Newton School of Technology
Rishihood University
Sonipat, Haryana, India

Design and Implementation of a Direct-Mapped Cache Controller

Phase I Project Report

Project Title: Direct-Mapped Cache Controller
Team Name: 5bit registers **Section:** C

Team Members

Name	Enrollment No.	Email
Krishna	2401010237	krishna.2024@nst.rishihood.edu.in
Aditya Rana	2401010034	aditya.rana2024@nst.rishihood.edu.in
Rachit Gupta	2401010363	rachit.gupta2024@nst.rishihood.edu.in
Saumya Mishra	2401020057	saumya.mishra2024@nst.rishihood.edu.in
Nishtha Gupta	2401010303	nishtha.gupta2024@nst.rishihood.edu.in

14 November 2026

Index

	Abstract and Index Terms	1
I.	Introduction	1
II.	Literature Review	1
III.	Proposed Solution	2
	A. System Architecture	2
	B. Cache Organization and Address Mapping	2
	C. Read and Write Operation	2
	D. Controller Microarchitecture	2
IV.	Implementation	3
	A. Module Hierarchy	3
	B. Main-Memory Model and Performance Monitor	3
	C. Top-Level Interface	3
V.	Verification and Results	4
	A. Verification Methodology	4
	B. Unit-Level Results	4
	C. Integrated Results and Waveform Scenarios	4
	D. Waveform Timing Analysis	4
	E. Integrated Access Trace	5
	F. Requirements Traceability	5
VI.	Performance Analysis	5
	A. Hit and Miss Rates	5
	B. Measured Completion Latency	5
	C. Average Access Time	5
	D. Trace-Driven Benchmark	6
VII.	Discussion and Limitations	8
VIII.	Alternative Organizations and Synthesis Results	8
IX.	Individual Contributions	8
X.	Conclusion and Future Work	9
	Appendix A: Reproducing the Results	9
	References	9

List of Figures

Fig. 1	Block-level architecture of the integrated direct-mapped cache	2
Fig. 2	32-bit byte-address partition used by the decoder	2
Fig. 3	Controller FSM paths	4
Fig. 4	Waveform: cold read miss followed by a read hit	6
Fig. 5	Waveform: write hit, write miss and read hit	6
Fig. 6	Analytical average read latency versus hit rate	7
Fig. 7	Analytical and measured AMAT for the benchmark workloads	7

List of Tables

Table I	Cache configuration	2
Table II	Worked address-decoding examples	2
Table III	Controller states and actions	3
Table IV	RTL modules and source files	3
Table V	`cache_top` ports	3
Table VI	Unit-level verification results	4
Table VII	Isolated waveform scenarios	5
Table VIII	Performance counters: expected vs. actual	5
Table IX	Classified access trace of the integrated sequence	6
Table X	Requirement-to-evidence mapping	7
Table XI	Measured completion latency per access type	7
Table XII	Benchmark workloads and simulated hit/miss counts	7
Table XIII	Trace benchmark results (analytical estimates)	7

Design and Implementation of a Direct-Mapped Cache Controller

Krishna
Enroll. No. 2401010237
krishna.2024@nst.rishihood.edu.in

Aditya Rana
Enroll. No. 2401010034
aditya.rana2024@nst.rishihood.edu.in

Rachit Gupta
Enroll. No. 2401010363
rachit.gupta2024@nst.rishihood.edu.in

Saumya Mishra
Enroll. No. 2401020057
saumya.mishra2024@nst.rishihood.edu.in

Nishtha Gupta
Enroll. No. 2401010303
nishtha.gupta2024@nst.rishihood.edu.in

Team "5bit registers", Section C
Newton School of Technology, Rishihood University
Sonipat, Haryana, India

***Abstract*— This paper presents the design and Phase I verification of a 32-bit direct-mapped cache subsystem that reduces average memory-access latency by exploiting temporal and spatial locality. The cache holds 1 KiB as 64 lines of 16-byte blocks (four 32-bit words per line), with a 22-bit tag, 6-bit index and 4-bit offset, and uses write-through and write-allocate policies. The modular Verilog RTL comprises an address decoder, cache storage, an FSM controller, a behavioral main-memory model and a performance monitor. Verification records 374 decoder checks, unit tests for every module, and 39 integrated checks passing with no failures over a 26-access directed sequence (13 hits, 13 misses), supported by eight waveform scenarios. Waveform measurements give completion latencies of 2, 11, 9 and 18 cycles for a read hit, read miss, write hit and write miss. Four read workloads replayed through the RTL give hit rates of 80% (sequential), 93.75% (repeated block), 0% (conflict thrashing) and 35.9% (seeded random), with measured average memory access times of 3.80, 2.56, 11.00 and 7.77 cycles; a Python model of the same mapping reproduces every hit and miss count. Synthesized for a Lattice ECP5-85F, the cache uses 18,851 LUTs, 10,978 flip-flops and one block RAM and closes timing at 61.1 MHz after place-and-route.**

***Index Terms*—Cache memory, direct mapping, cache controller, Verilog HDL, write-through, write-allocate, finite-state machine, memory hierarchy.**

I. Introduction

The processor–memory performance gap motivates the use of cache memory: a small, fast storage structure placed between a processor and a larger, slower main memory. Caches exploit *temporal locality*, in which recently accessed data is likely to be accessed again, and *spatial locality*, in which nearby addresses are likely to be accessed soon. A cache keeps recently used memory blocks so that subsequent requests can be served without accessing main memory [1], [2].

This project develops a modular direct-mapped cache subsystem in Verilog HDL [5]. In direct mapping, each main-memory block is assigned to exactly one cache line. The line index selects the candidate line, while a stored tag and a valid bit determine whether the requested block is present [1], [2]. Direct mapping offers a simple lookup and replacement mechanism, although blocks that map to the same line can evict one another.

The Phase I objectives are to (i) specify the microarchitecture and the interface contract between modules, (ii) develop modular RTL, (iii) verify functional behavior with directed self-checking testbenches, (iv) expose hardware counters that classify every completed processor access as a hit or a miss, and (v) evaluate locality and conflict behavior with a reproducible trace-driven benchmark. The remainder of this paper is organized as follows. Section II reviews background literature, Section III describes the proposed architecture, Section IV details the implementation and its interfaces, Section V reports verification results including waveform timing, Section VI analyzes performance from measured latencies and a trace-driven benchmark, Section VII discusses design decisions and limitations, and Sections VIII–X cover alternatives, contributions and conclusions. Appendix A explains how to reproduce the results.

II. Literature Review

Classical computer-architecture literature describes cache organization in terms of placement, identification and replacement [1], [4]. In a direct-mapped organization, the index selects one line and the tag comparison determines whether that line contains the requested memory block. Only one tag comparison is required, which simplifies both lookup and replacement logic [1].

Arm's architecture documentation explains that a direct-mapped cache maps each main-memory location to one cache location. The address is divided into tag, index and offset fields: the index chooses a line, the tag identifies the resident block, and the offset selects the requested data within the block [2]. The IIT Kharagpur Virtual Labs compare direct-mapped, fully associative and set-associative organizations; direct mapping is straightforward because each memory block has one permitted location, but its rigid placement can cause

TABLE I
CACHE CONFIGURATION

Parameter	Value
Address width	32 bits (byte addressed)
Capacity	1 KiB (1024 bytes)
Mapping	Direct-mapped
Lines × block size	64 × 16 bytes
Words per line	4 × 32-bit
Tag / index / offset	22 / 6 / 4 bits
Access granularity	Aligned 32-bit words
Write policy	Write-through, write-allocate
Main-memory latency (model)	5 cycles (configurable)

conflict misses when frequently used blocks compete for the same line [3].

Standard write policies include *write-through*, in which a write updates both the cache and the next memory level, and *write-back*, in which modified data may remain in the cache until eviction [4]. This project uses write-through to keep main memory consistent with cache contents and *write-allocate* to fetch a block on a write miss before updating the selected word. The combination removes the need for dirty bits and simplifies the prototype, at the cost of requiring memory completion for every write.

III. PROPOSED SOLUTION

The proposed system is a 1 KiB, 32-bit, direct-mapped cache with 64 lines and 16-byte blocks. Each line holds a valid bit, a 22-bit tag and 128 data bits (four 32-bit words). The address decoder extracts the tag, line index and word offset. Cache storage holds tags, valid bits and data blocks. The controller coordinates processor requests, cache lookups, refills, writes and memory handshakes. The performance monitor counts completed processor accesses and classifies them as hits or misses. Table I summarizes the configuration.

A. System Architecture

Fig. 1 shows the logical module hierarchy. The processor interface supplies a request, an operation type, an address and write data. The controller latches an accepted request and uses the decoder and cache storage for lookup. On a miss, it requests a 16-byte block from main memory; on writes, it propagates the selected 32-bit word to memory. Completion events are counted by the performance monitor.

B. Cache Organization and Address Mapping

The cache capacity is 1024 bytes. With 16 bytes per line, the number of lines is $1024/16 = 64$, requiring six index bits. Each block contains four 32-bit words, requiring two word-select bits, and a 16-byte block requires four offset bits in a byte-addressed system. The remaining $32 - 6 - 4 = 22$ bits form the tag:

$$\underbrace{A[31{:}10]}_{\text{22-bit tag}}\ \underbrace{A[9{:}4]}_{\text{6-bit index}}\ \underbrace{A[3{:}2]}_{\text{2-bit word}}\ \underbrace{A[1{:}0]}_{\text{2-bit byte}} \tag{1}$$

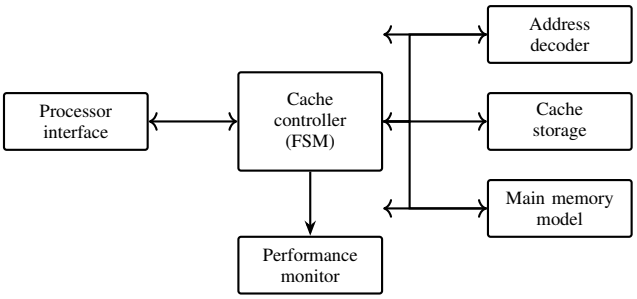


Fig. 1. Block-level architecture of the integrated direct-mapped cache.

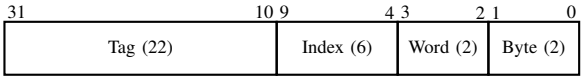


Fig. 2. 32-bit byte-address partition used by the decoder.

TABLE II
WORKED ADDRESS-DECODING EXAMPLES

Address	Tag	Index	Word	Block base
0x00000000	0	0	0	0x000
0x00000004	0	0	1	0x000
0x00000400	1	0	0	0x400
0x00000804	2	0	1	0x800
0x00000060	0	6	0	0x060
0x00000460	1	6	0	0x460

as illustrated in Fig. 2. The current processor interface is word oriented and assumes word-aligned accesses ($A[1{:}0] = 00$). Block data are ordered so that `block_data[31:0]` holds the lowest-address word, followed by `[63:32]`, `[95:64]` and `[127:96]`.

Table II gives worked examples taken from the integrated test. Addresses `0x000`, `0x400` and `0x804` all map to index 0 but carry different tags (0, 1 and 2), so they evict one another; this is how the conflict-replacement behavior is provoked in simulation.

C. Read and Write Operation

Read hit: the controller returns the selected word from the indexed line. *Read miss:* it requests the aligned 16-byte block from main memory, waits for the memory-ready response, installs the returned block and tag, marks the line valid and returns the requested word. *Write hit:* the selected cache word is updated and the same word is written to main memory; completion is reported after the memory acknowledgement. *Write miss:* the controller first fetches and installs the containing block, updates the selected word, writes that word through to main memory and then completes the request. Because every write is propagated to memory, no dirty bits are required.

D. Controller Microarchitecture

The controller is an FSM with eight states (Fig. 3): `IDLE`, `LOOKUP`, `REFILL_WAIT`, `REFILL_INSTALL`, `READ_MISS_RETURN`, `WRITE_HIT_WAIT`,

TABLE III
CONTROLLER STATES AND ACTIONS

Code	State	Action
0	IDLE	Wait for cpu_req; latch address, operation and write data
1	LOOKUP	Evaluate hit/miss; complete a read hit; start a write hit or a refill
2	REFILL_WAIT	Hold block request until mem_ready
3	REFILL_INSTALL	Install 128-bit block, tag and valid bit
4	READ_MISS_RETURN	Return the requested word; report a miss
5	WRITE_HIT_WAIT	Wait for write-through acknowledgement; report a hit
6	WRITE_MISS_UPDATE	Update the selected word; issue write-through
7	WRITE_MISS_WAIT	Wait for acknowledgement; report a miss

TABLE IV
RTL MODULES AND SOURCE FILES

Module	File
Address decoder	rtl/address_decoder.v
Cache storage	rtl/cache_storage.v
Cache controller	rtl/cache_controller.v
Performance monitor	rtl/performance_monitor.v
Main-memory model	memory/main_memory_model.v
Integrated top level	rtl/cache_top.v

WRITE_MISS_UPDATE and WRITE_MISS_WAIT. In IDLE the controller captures a new request. LOOKUP determines hit or miss: a read hit returns data directly, while a write hit updates the cache word and issues a memory write. A miss requests a block refill and waits for memory. After refill the block is installed; a read miss then returns the selected word, whereas a write miss updates the selected word and issues a write-through transaction.

The processor protocol allows one outstanding request. Requests presented while the controller is busy are not queued, and the requester must deassert cpu_req for at least one rising clock edge between transactions. The controller produces a one-cycle cpu_ready completion pulse. The registered events access_complete, hit_event and miss_event identify completed requests; exactly one of hit or miss accompanies each completion.

Table III lists the state encodings, which are exposed on the controller_state output for waveform inspection.

IV. IMPLEMENTATION

A. Module Hierarchy

The implemented hierarchy contains five leaf modules (about 650 lines of Verilog in total) integrated by cache_top; Table IV lists them. All sequential modules share clk and a synchronous active-high reset.

The *decoder* is combinational: it extracts the address fields and performs a 22-bit tag comparison qualified by the stored valid bit, so that

$$\text{hit} = \text{stored_valid} \wedge (\text{stored_tag} = A[31{:}10]). \quad (2)$$

Cache storage contains 64 lines, each with a valid bit, a 22-bit tag and 128 data bits; lookup is combinational, refill and word updates are synchronous, reset invalidates every line, and a refill takes priority over a simultaneous word write. The *controller* is sequential RTL that latches the request address, operation and write data on acceptance and drives storage, memory and monitor interfaces with registered one-cycle pulses.

TABLE V
CACHE_TOP PORTS

Signal	Dir.	Bits	Description
clk, reset	in	1	Clock; sync. reset
cpu_req	in	1	Request (one outstanding)
cpu_write	in	1	1 = write, 0 = read
cpu_addr	in	32	Aligned word address
cpu_wdata	in	32	Write data
cpu_ready	out	1	One-cycle completion
cpu_rdata	out	32	Read data with ready
total_accesses	out	32	Completed accesses
total_hits	out	32	Hit counter
total_misses	out	32	Miss counter
controller_state	out	4	FSM state (debug)

B. Main-Memory Model and Performance Monitor

The main-memory model is behavioral and intended for simulation. It contains 4096 32-bit words (16 KiB), initializes all words to zero, optionally loads hexadecimal contents through $readmemh, supports configurable latency (default five cycles), returns four consecutive words on a block read and supports a 32-bit word write. Reset clears protocol state but does not erase memory contents. The initialization file is a simulation feature and is not part of a synthesizable physical memory macro.

The synthesizable performance monitor uses three 32-bit counters (total_accesses, total_hits, total_misses) with synchronous reset. It increments only when access_complete is high and exactly one of hit_event or miss_event is asserted, so that

$$\text{total_accesses} = \text{total_hits} + \text{total_misses} \quad (3)$$

holds between counter wraparounds. Internal refill and write-through memory operations are not counted as separate processor accesses.

C. Top-Level Interface

The integrated module cache_top exposes the processor-side interface and the performance counters, and parameters MEM_WORDS (4096) and MEM_LATENCY (5) for the embedded memory model (Table V). Internally, the controller drives the memory with a request, a write-enable, a byte address, 32-bit write data and receives a ready strobe and a 128-bit block.

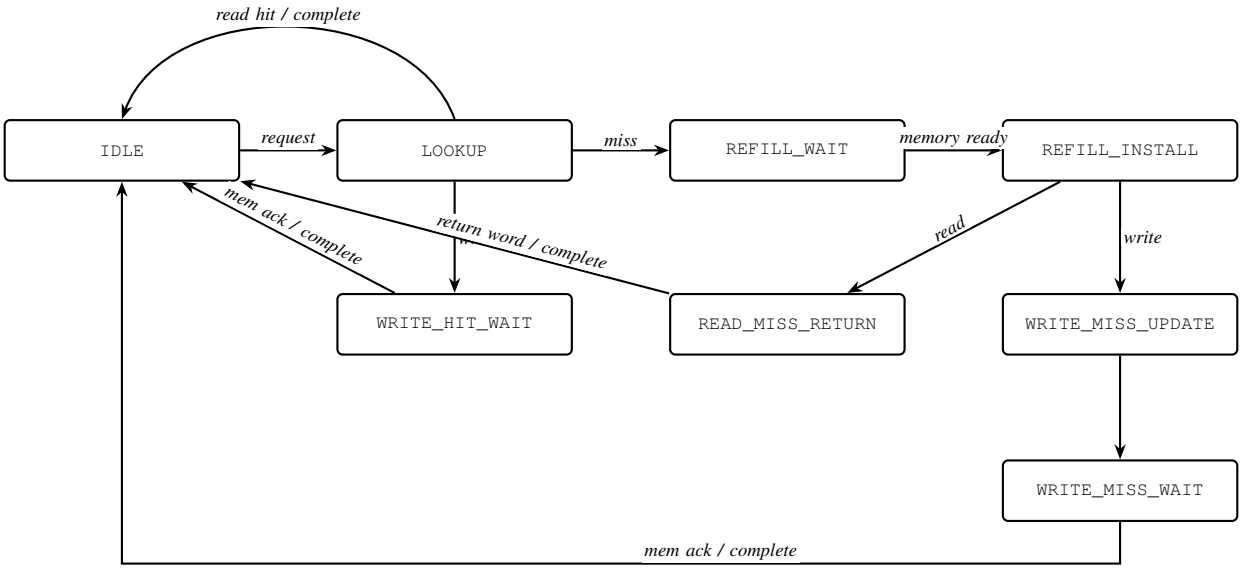


Fig. 3. Controller FSM paths for read hits, misses and write-through operations. The controller returns to `IDLE` after the completion pulse; write operations complete after the memory acknowledgement.

V. Verification and Results

A. Verification Methodology

Each module has a self-checking Verilog testbench, and a top-level testbench instantiates the real `cache_top` with the actual decoder, storage, controller, memory and monitor. Simulations are compiled with Icarus Verilog [6] (`iverilog -g2012 -Wall`) and executed with `vvp`. In the integrated test, memory is seeded with the deterministic pattern `0x600d0000 + word_index`, so every returned word can be checked against a known value. The regression is run from the repository root with a single script (`sim/run_krishna_tests.sh`), which runs all unit benches, the integrated bench and the eight waveform scenarios. The final run completed with every suite passing.

B. Unit-Level Results

Table VI summarizes the unit-level regression. The address-decoder bench alone executes ten suites and 374 checks, including a full 64-index sweep, all four word offsets, valid/tag combinations, walking-one/zero isolation of all 22 tag bits and 200 randomized vectors checked against a reference model. The controller bench exercises cold miss and four-word refill, repeated hits, conflicting-tag replacement, write hit, write miss and a dropped busy request, and ends with 13 accesses classified as 7 hits and 6 misses. The memory bench confirms a four-word block read, exact-word write, read-after-write and a measured response wait of five cycles. The memory-initialization bench confirms that `$readmemh` contents appear in ascending address order; Icarus emits a sparse-file warning because the sample file holds four words rather than 4096, and the remaining locations stay zero.

TABLE VI
Unit-Level Verification Results

Testbench	Result	Status
Address decoder	10 suites, 374 checks, 0 errors	Pass
Cache storage	Reset, refill, word select/update, refill priority	Pass
Controller	13 accesses: 7 hits, 6 misses	Pass
Main memory	Block read, write, read-after-write, 5-cycle latency	Pass
Memory init.	`$readmemh` order verified	Pass
Perf. monitor	10 tests, invariant (3) held	Pass

C. Integrated Results and Waveform Scenarios

The combined integrated sequence passed **39 checks with 0 failures** over 26 completed processor accesses, with the monitor reporting 13 hits and 13 misses, matching the expected counts exactly. In addition, eight isolated runs reset the design and counters and dump a VCD waveform each (Table VII). Every run passed and its counters matched expectation (Table VIII). The VCD files include clock/reset, processor request and response signals, controller state, performance counters, memory request/response/address/block data, and cache refill and word-write controls.

The conflict scenario demonstrates the main weakness of direct mapping: addresses `0x0` and `0x400` share index 0 but differ in tag, so each evicts the other. The write-through scenario confirms that a written word appears in both the cache and main memory, that the other three words of the block are unchanged, and that a write miss allocates the block.

D. Waveform Timing Analysis

The committed VCD files were post-processed to extract the controller state, handshake and completion signals. Fig. 4

TABLE VII
ISOLATED WAVEFORM SCENARIOS

Scenario	Behavior demonstrated	Chk.
reset_invalid_lines	Valid bits clear; first read misses	5
cache_miss	Cold miss returns memory data	5
cache_hit	Repeat access hits	6
consecutive_requests	Protocol-legal back-to-back reads	7
controller_busy	Busy request dropped, not queued	6
block_refill_order	Words 0–3 in ascending order	9
conflict_replacement	Same-index eviction, no stale data	9
write_through	Write hit/miss visible in cache and memory	12

TABLE VIII
PERFORMANCE COUNTERS: EXPECTED VS. ACTUAL

Run	Acc.	Hits	Miss
Full combined sequence	26	13	13
Reset / invalid lines	1	0	1
Cache miss	1	0	1
Cache hit	2	1	1
Consecutive requests	3	2	1
Controller busy	2	0	2
Four-word refill order	4	3	1
Conflict replacement	4	1	3
Write-through	4	2	2

Expected and actual values were identical in every run.

shows the cache_hit scenario. The first read is a cold miss: the controller captures the request, performs the lookup, issues mem_req, remains in REFILL_WAIT for seven cycles (the configured five-cycle memory latency plus two cycles of request/response handshake), installs the block, and returns the word with cpu_ready and miss_event. The repeated read then follows the short IDLE → LOOKUP → IDLE path and raises hit_event one cycle after the lookup.

Fig. 5 shows write-through behavior. On the write hit, a memory write (mem_req with mem_write_en) is issued from LOOKUP and the controller waits in WRITE_HIT_WAIT until mem_ready; only then are cpu_ready and hit_event asserted. On the write miss the controller first refills the block, then issues the write-through and waits in WRITE_MISS_WAIT, completing with miss_event. Exactly one of hit_event or miss_event accompanies each completion in all recorded scenarios.

E. Integrated Access Trace

Table IX lists all 26 accesses of the combined integrated sequence (reads **R**, writes **W**; hit **H**, miss **M**). It contains 22 reads and 4 writes: 11 read hits, 11 read misses, 2 write hits and 2 write misses. All results match the expected classification enforced by the testbench, and the monitor's final counters (26/13/13) agree with the trace.

F. Requirements Traceability

Table X maps each design requirement to the evidence that exercises it.

VI. PERFORMANCE ANALYSIS

A. Hit and Miss Rates

For the integrated directed sequence, 26 completed processor accesses were recorded: 13 hits and 13 misses. The sequence-specific rates are

$$H = \frac{13}{26} \times 100 = 50\%, \qquad M = 50\%. \tag{4}$$

These values describe the test sequence only; the sequence deliberately mixes cold misses, conflicts and writes, and is not a workload benchmark; the trace-driven benchmark of Section VI-D addresses locality.

B. Measured Completion Latency

Latency L is defined as the number of rising clock edges from the request-capture edge (IDLE→LOOKUP) up to and including the edge on which cpu_ready is registered. It was measured from the recorded waveforms with the 10 ns testbench clock and the default five-cycle memory latency, and was identical in every scenario that exercised the same access type (Table XI).

The miss penalty for reads is therefore $11 - 2 = 9$ cycles, dominated by the seven-cycle refill wait. Because the cache is write-through, a write hit still costs 9 cycles (4.5× a read hit), and a write miss costs 18 cycles since it pays for both the refill and the memory write.

C. Average Access Time

The standard model for the effect of the cache on latency is [4]

$$\mathrm{AMAT} = t_{\mathrm{hit}} + m \cdot t_{\mathrm{penalty}}. \tag{5}$$

With the measured read latencies, $t_{\mathrm{hit}} = 2$ and $t_{\mathrm{penalty}} = 9$ cycles, so for reads

$$\mathrm{AMAT}_{\mathrm{read}} = 2 + 9(1 - H) = 11 - 9H \text{ cycles}, \tag{6}$$

plotted in Fig. 6. The 22 reads of the integrated sequence have $H = 11/22 = 50\%$, giving 6.5 cycles. Weighting all 26 accesses by their measured latency from Table XI gives a total service time of

$$11{\cdot}2 + 11{\cdot}11 + 2{\cdot}9 + 2{\cdot}18 = 197 \text{ cycles}, \tag{7}$$

an average of $197/26 \approx 7.6$ cycles per access. This figure is derived from the measured per-type latencies rather than from a single end-to-end simulation, and it excludes idle gaps between requests.

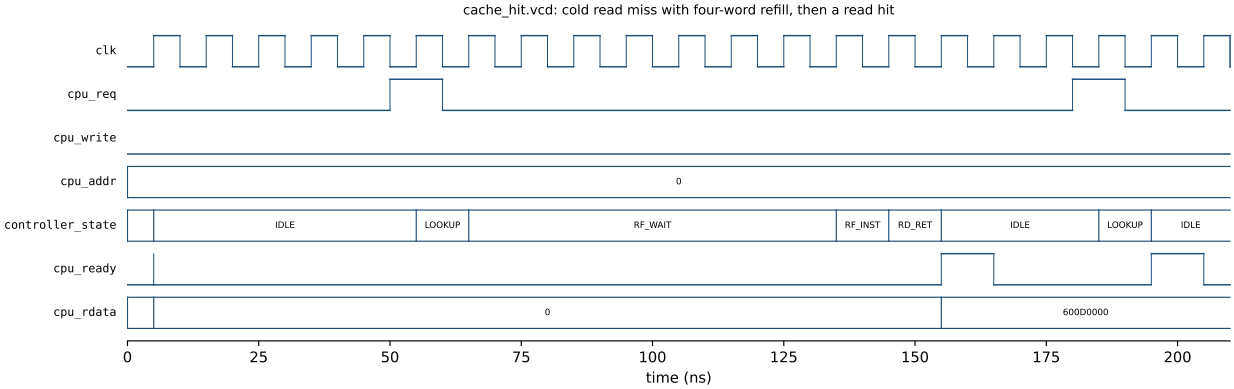


Fig. 4. Waveform extracted from `cache_hit.vcd`: a cold read miss with four-word refill followed by a read hit.

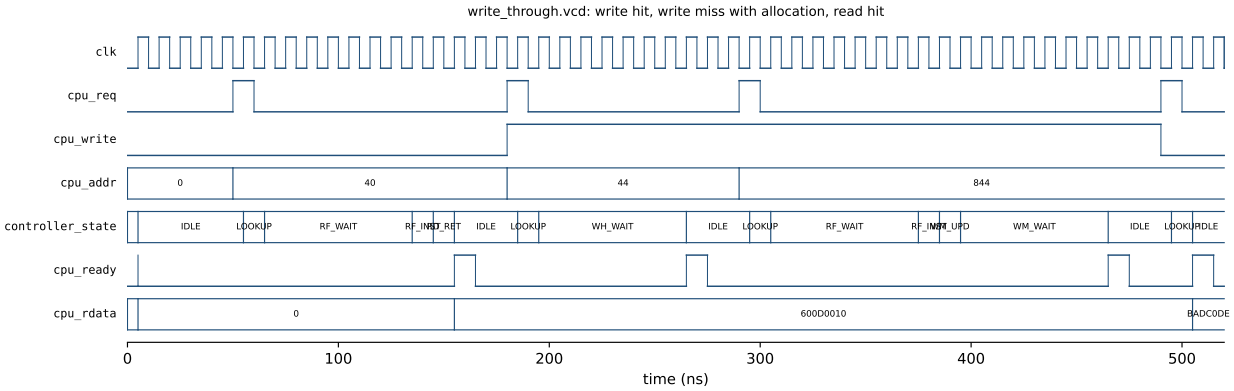


Fig. 5. Waveform extracted from `write_through.vcd`: write hit, write miss with allocation, and a read hit to the allocated line.

TABLE IX
CLASSIFIED ACCESS TRACE OF THE INTEGRATED SEQUENCE

#	Op	Address	Idx	Res.	#	Op	Address	Idx	Res.
1	R	0x000	0	M	14	R	0x050	5	M
2	R	0x000	0	H	15	R	0x03C	3	M
3	R	0x004	0	H	16	R	0x030	3	H
4	R	0x400	0	M	17	R	0x034	3	H
5	R	0x000	0	M	18	R	0x038	3	H
6	W	0x004	0	H	19	R	0x060	6	M
7	R	0x004	0	H	20	R	0x460	6	M
8	W	0x804	0	M	21	R	0x460	6	H
9	R	0x804	0	H	22	R	0x060	6	M
10	R	0x010	1	M	23	R	0x040	4	M
11	R	0x014	1	H	24	W	0x044	4	H
12	R	0x018	1	H	25	W	0x844	4	M
13	R	0x020	2	M	26	R	0x844	4	H

D. Trace-Driven Benchmark

Methodology. To evaluate locality beyond the directed tests, a trace-driven functional model of the cache was written in Python [7]. It is an architecture-level model, not an RTL simulation, and mirrors the RTL configuration of 64 lines and 16-byte blocks. For each word-aligned byte address, the block number is $\lfloor A/16 \rfloor$, the index is the block number modulo 64 and the tag is $\lfloor \text{block}/64 \rfloor$. Each workload starts with all valid bits clear; a hit requires a valid indexed line with a matching tag, and a miss installs the requested tag. The hit rate is hits divided by reads, and the analytical average memory access time is Eq. (5) with the idealized assumptions $t_{\text{hit}} = 1$ cycle and $t_{\text{penalty}} = 5$ cycles (the configured main-memory response latency). The traces are read-only, so write-through behavior is not exercised. The trace dataset [8] contains the

TABLE X
REQUIREMENT-TO-EVIDENCE MAPPING

Requirement	Evidence
64-line index mapping	Decoder index sweep (0–63)
Hit = valid ∧ tag match	Decoder tests 4–7; tag-bit isolation
Reset invalidates lines	Storage reset test; `reset_invalid_lines`
Four-word block order	`block_refill_order`; memory block read
Direct-mapped replacement	`conflict_replacement`; trace #4–5, #20–22
Write-through, write-allocate	`write_through`; trace #6, #8, #24–25
One outstanding request	`controller_busy` (dropped request)
Hit/miss accounting	Monitor tests; counters 26/13/13

TABLE XI
MEASURED COMPLETION LATENCY PER ACCESS TYPE

Access	Cycles	ns	State path
Read hit	2	20	`LOOKUP`
Read miss	11	110	`LOOKUP`, `REFILL_WAIT`, `REFILL_INSTALL`, `READ_MISS_RETURN`
Write hit	9	90	`LOOKUP`, `WRITE_HIT_WAIT`
Write miss	18	180	`LOOKUP`, `REFILL_WAIT`, `REFILL_INSTALL`, `WRITE_MISS_UPDATE`, `WRITE_MISS_WAIT`

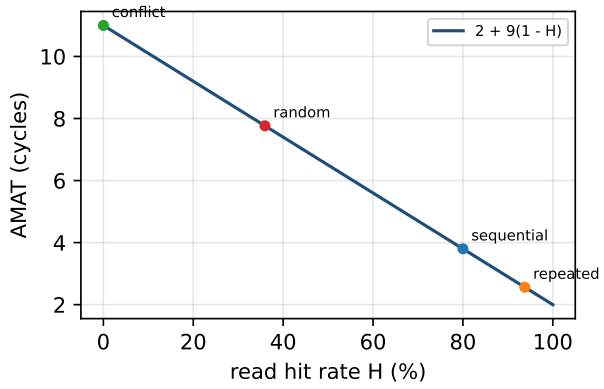


Fig. 6. Analytical average read latency versus read hit rate, using the measured 2-cycle hit and 11-cycle miss latencies.

four workloads in Table XII; the random workload is generated by `analysis/gen_random_trace.py` with seed 2026.

RTL replay. The same traces are also replayed through `cache_top` by `sim/testbenches/tb_trace_replay.v`. Each workload runs from reset, every memory word is preloaded with a distinct pattern so that a wrong word or stale line shows up as a data error, and the cycles from the request edge to `cpu_ready` are counted for every access. Hit and miss counts are read from the hardware performance counters. The measured AMAT is total cycles divided by

TABLE XII
BENCHMARK WORKLOADS AND SIMULATED HIT/MISS COUNTS

Workload	Access pattern	Reads	Hits	Misses
Sequential	Words `0x00–0x3C` once, then `0x00–0x0C` again	20	16	4
Repeated block	Four words at `0x100–0x10C`, repeated four times	16	15	1
Conflict thrashing	`0x000` and `0x400` alternating (same index, different tags)	8	0	8
Random	64 uniform word addresses in `0x000–0x3FC`, seed 2026	64	23	41

TABLE XIII
TRACE BENCHMARK RESULTS

Workload	Hit rate	Analytical	Measured (RTL)
Sequential	80%	2.00 cycles	3.80 cycles
Repeated block	93.75%	1.3125 cycles	2.5625 cycles
Conflict thrashing	0%	6.00 cycles	11.00 cycles
Random	35.94%	4.20 cycles	7.77 cycles

Analytical: $1+5M$ (1-cycle hit, 5-cycle miss penalty). Measured: total cycles over accesses from the RTL replay, which equals $2+9M$.

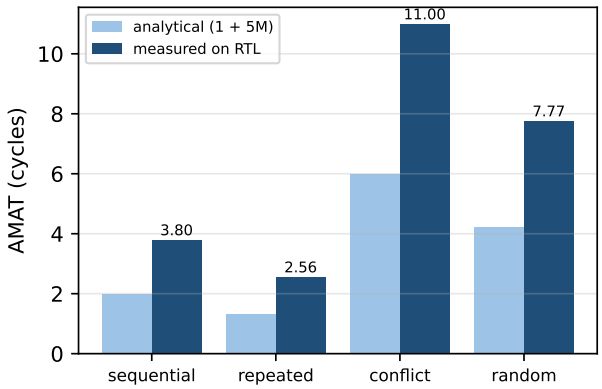


Fig. 7. Analytical and RTL-measured AMAT for the four benchmark workloads.

accesses.

Results. The sequential trace misses once per 16-byte block (four cold misses) and then hits on the three remaining words of each block and on the re-read of the first block, illustrating spatial and temporal locality. The repeated-block trace incurs a single cold miss and 15 hits. The conflict trace maps both addresses to index 0, so each access evicts the other and every read misses; this is the pathological case for direct mapping. The random trace touches a working set the size of the cache, so it starts cold and warms as lines are reused, ending at a 35.9% hit rate. The RTL replay produced exactly the same hit and miss counts as the Python model on all four traces, and every returned word matched memory. Table XIII and Fig. 7 compare the analytical and measured AMAT.

Interpretation and limits. The analytical column uses the

textbook convention and must *not* be read as cycle-accurate: it omits controller state transitions and refill-installation cycles that the waveforms show (a read hit takes 2 cycles and a read miss 11, not 1 and 6). The measured column comes from replaying each trace through `cache_top`, and it agrees with Eq. (6) evaluated on the measured hit and miss latencies. The traces are small, deliberately constructed patterns that demonstrate spatial locality, temporal locality and conflict misses; they are not a real application workload. No cache-less baseline, CPI, throughput or speedup has been measured, so none is claimed.

VII. Discussion and Limitations

Design decisions. Direct mapping was chosen because it needs one tag comparison and no replacement-policy state; write-through with write-allocate removes dirty bits and eviction write-backs, so a replaced line can simply be overwritten. A single-outstanding-request protocol keeps the controller to eight states and makes every access end in exactly one hit or miss event.

Limitations. (i) Only aligned 32-bit word accesses are supported; byte and half-word writes would require byte enables. (ii) Requests presented while the controller is busy are dropped, so the processor must retry. (iii) Without a write buffer, every write stalls for the memory acknowledgement, which the latency measurements show to be the dominant cost for writes. (iv) The memory is a behavioral simulation model; a synthesized design needs an on-chip block RAM or an external memory controller with its own timing. (v) The 10 ns testbench clock is faster than the 61.1 MHz the design reaches after place-and-route (Section VIII). (vi) The directed sequence demonstrates correctness, and the trace benchmark uses small constructed read-only traces, so neither establishes real-workload performance. (vii) The 32-bit counters wrap after 2^{32} accesses.

Conflict misses. The trace shows three-way contention at index 0 (addresses `0x000`, `0x400`, `0x804`) and two-way contention at indices 4 and 6. The conflict-thrashing benchmark shows the worst case: a 0% hit rate for two alternating addresses that share an index. A two-way set-associative organization would hold such pairs simultaneously, at the cost of a second comparator and way-selection logic [1], [3].

VIII. Alternative Organizations and Synthesis Results

Compared with fully associative placement, direct mapping requires only one candidate line and one tag comparison, reducing lookup and replacement complexity; however, it can incur conflict misses when active blocks share an index. Set-associative caches relax this placement restriction but require multiple tag comparisons and way-selection logic [1], [3]. Phase I selected direct mapping to keep the RTL hierarchy and FSM manageable. No quantitative hardware comparison against another implemented cache organization is available.

TABLE XIV
ECP5-85F Synthesis and Place-and-Route Results

Resource	Used	Available	%
LUT4	18,851	83,640	22
Flip-flops	10,978	83,640	13
Block RAM (DP16KD)	1	208	<1
I/O	201	365	55
Post-route $f_{\max}$: **61.1 MHz** (100 MHz target not met)			

Synthesis. The design was synthesized with Yosys 0.71 and placed and routed with nextpnr-ecp5 (seed 1, 100 MHz target) for a Lattice ECP5-85F in the CABGA756 package; scripts and logs are in `synth/`. For place-and-route the behavioral memory is shrunk to 64 words so it becomes a small stand-in; the cache is unchanged. Table XIV lists the results.

A technology-independent run with the memory as a black box gives 10,136 flip-flops, of which 8,192 hold the data array and 1,408 the tags. The data array reads a whole block in one cycle, so it stays in flip-flops instead of block RAM; the tag array is the only structure inferred as RAM. The critical path starts at that RAM's 5.8 ns clock-to-output and runs through the 22-bit tag compare and the controller next-state logic. Registering the hit decision or banking the data array into four 64×32 RAMs would shorten it. An iCE40 HX8K was also tried: the design needs 16,467 LUTs against 7,680 available, so it does not fit.

IX. Individual Contributions

The following division is based on module ownership and artifacts recorded in the repository.

- **Krishna:** defined the architecture and RTL interface contract; implemented the controller FSM, cache-top wiring, request/response protocol, refill sequencing and write policies.
- **Aditya Rana:** implemented the 32-bit address decoder and 22-bit tag comparator; added a self-checking testbench with ten suites and 374 checks, a runner, waveform output and documentation.
- **Rachit Gupta:** implemented the 64-line cache storage with tags, valid bits, four-word refill, lookup and selected-word updates; added the cache-storage unit testbench.
- **Saumya Mishra:** implemented the behavioral memory model with configurable latency, block reads, word writes and initialization; implemented the synthesizable performance monitor and its unit tests.
- **Nishtha Gupta:** added the integrated cache-top testbench, edge-case verification, eight isolated VCD scenarios and the consolidated verification report.
- **Trace benchmark and synthesis:** Saumya Mishra added the read-trace dataset and the analytical model; Krishna added the random workload, the RTL trace-replay testbench and the ECP5 synthesis flow.

X. Conclusion and Future Work

This Phase I project delivers a 1 KiB direct-mapped cache with four-word blocks, write-through/write-allocate policies, a behavioral main memory and performance counters. Unit tests cover the decoder, storage, controller, memory and monitor. The integrated test passed 39 checks with no failures across 26 accesses (13 hits, 13 misses), and eight isolated VCD scenarios supplement the directed tests. Waveform measurements show completion latencies of 2, 11, 9 and 18 cycles for a read hit, read miss, write hit and write miss. Four read workloads replayed through the RTL gave hit rates of 80%, 93.75%, 0% and 35.9% for sequential, repeated-block, conflict-thrashing and random patterns, with measured AMAT of 3.80, 2.56, 11.00 and 7.77 cycles. On a Lattice ECP5-85F the cache uses 22% of the LUTs and reaches 61.1 MHz. These results confirm the expected behavior: strong spatial and temporal locality benefit, and a worst case when addresses share an index.

Further work includes moving the data array into block RAM to raise $f_{\max}$, benchmarking with real application traces including writes, and a cache-less baseline to quantify CPI and speedup. Byte and unaligned accesses are outside the current prototype contract and are natural extensions, as are a set-associative variant and a write-back policy for comparison.

Appendix A
Reproducing the Results

The project is hosted at https://github.com/CosmicMagnetar/MCA_Capstone_Cache. Icarus Verilog [6] must be installed. From the repository root:

- `bashsim/run_krishna_tests.sh` compiles and runs every unit testbench, the integrated bench and all eight waveform scenarios, and ends with "ALL PHASE I UNIT, INTEGRATION, AND WAVEFORM TESTS COMPLETED".
- `bashsim/run_cache_top_verification.sh` runs only the integrated and waveform tests.
- `python3analysis/cache_trace_benchmark.py` runs the trace benchmark [7] on `sim/traces/phase1_read_traces.csv` [8]; `--hit-time` and `--miss-penalty` override the 1- and 5-cycle assumptions.
- `bashsim/run_trace_benchmark.sh` replays every workload through the RTL and writes `docs/results/rtl_trace_results.csv`.
- `bashsynth/run_synthesis.sh` reruns synthesis and place-and-route (Yosys and nextpnr-ecp5).
- `python3report/make_figures.py` regenerates the four figures from the VCDs and the trace results.
- A single scenario can be run by passing `+SCENARIO=<name>` to `vvp`, where `<name>` is one of the eight scenario names in Table VII.

Source files are in `rtl/` and `memory/`, testbenches in `sim/testbenches/`, waveforms and the consolidated verification report in `sim/waveforms/`, and design notes in `docs/`.

Appendix B
Summary of Simulation Log

The final regression log reports: address decoder 374 checks, 0 errors; cache storage all tests passed; controller 13 accesses (7 hits, 6 misses); main memory block read, write, read-after-write and 5-cycle latency passed; memory initialization passed; performance monitor 10 of 10 tests passed; integrated `cache_top`: 39 run, 39 passed, 0 failed with counters 26/13/13; and all eight waveform scenarios passed with counters matching expectation.

References

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Waltham, MA, USA: Morgan Kaufmann, 2013.

[2] Arm Ltd., "Direct-mapped caches," in *Armv7-R Programmer's Guide*. [Online]. Available: https://support.arm.com/documentation/den0042/0100/Caches/Cache-architecture/Direct-mapped-caches. Accessed: Nov. 13, 2026.

[3] IIT Kharagpur Virtual Labs, "Direct mapped cache," *Computer Organization and Architecture*. [Online]. Available: https://coa-iitkgp.vlabs.ac.in/exp/direct-mapped-cache-design/theory.html. Accessed: Nov. 13, 2026.

[4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 4th ed. San Francisco, CA, USA: Morgan Kaufmann, 2007.

[5] *IEEE Standard for Verilog Hardware Description Language*, IEEE Std 1364-2005, 2006.

[6] S. Williams, "Icarus Verilog." [Online]. Available: https://steveicarus.github.io/iverilog/. Accessed: Nov. 13, 2026.

[7] Team 5bit registers, "cache_trace_benchmark.py: Trace-driven functional model of the 1 KiB direct-mapped cache," `analysis/` directory, MCA Capstone Cache repository, 2026. [Online]. Available: https://github.com/CosmicMagnetar/MCA_Capstone_Cache

[8] Team 5bit registers, "phase1_read_traces.csv: Sequential, repeated-block and conflict read traces," `sim/traces/` directory, MCA Capstone Cache repository, 2026. [Online]. Available: https://github.com/CosmicMagnetar/MCA_Capstone_Cache